ZYNTRA

Sistema de gestión de envíos · Tegucigalpa, Honduras

Documento de diseño

Primer parcial: Arquitectura y patrones

Autora	Tiareth Estrada · 529611
Materia	Ingeniería de Software II
Carrera	Ingeniería en Tecnologías de la Información y Negocios Digitales
Institución	Universidad Anáhuac Mayab
Fecha	4 de noviembre de 2026
Repositorio	github.com/TiarethE/zyntra · etiqueta v1.0

Contenido

1 Introducción

2 Arquitectura de Zyntra

3 Punto de partida y deuda técnica

4 Justificación de los patrones

5 Diseño para el rastreo en tiempo real

6 Comparativa UML: antes y después

7 Implementación y repositorio

8 Conclusiones y limitaciones

A Referencias

1 Introducción

Zyntra es el sistema con el que una paquetería con sede en Tegucigalpa, Honduras, registra sus envíos, calcula cuánto cuestan, sigue su recorrido y avisa al cliente cada vez que algo cambia.

La empresa recibe paquetes en sus sucursales, los acomoda en bodegas y ofrece tres servicios: **nacional**, **express** (entrega rápida dentro de Honduras) e **internacional**, que puede quedar retenido en aduana. Los clientes necesitan rastrear su paquete casi en tiempo real. Todos los montos se manejan en lempiras (HNL).

La primera versión del sistema (v0.1) metía todo en una sola clase. Este documento explica cómo se pagó esa deuda técnica con tres patrones de diseño, uno de cada familia, más el patrón Observer para el rastreo:

Factory Method · creacional
Un creador por tipo de envío que valida los datos antes de crearlo.

Decorator · estructural
El costo base más capas opcionales de seguro y manejo frágil.

State · comportamiento
Una clase por estado del envío, que conoce sus transiciones.

Observer · comportamiento
Un bus de eventos que avisa los cambios sin detener la operación.

Todo lo descrito aquí está implementado en el repositorio con la etiqueta **v1.0**. Los diagramas se escribieron a partir del código, y una prueba automática comprueba que cada clase, atributo y método dibujado existe en él. Las 48 pruebas del proyecto pasan.

2 Arquitectura de Zyntra

El sistema se organiza en cinco capas. Cada capa solo depende de las que están debajo de ella.

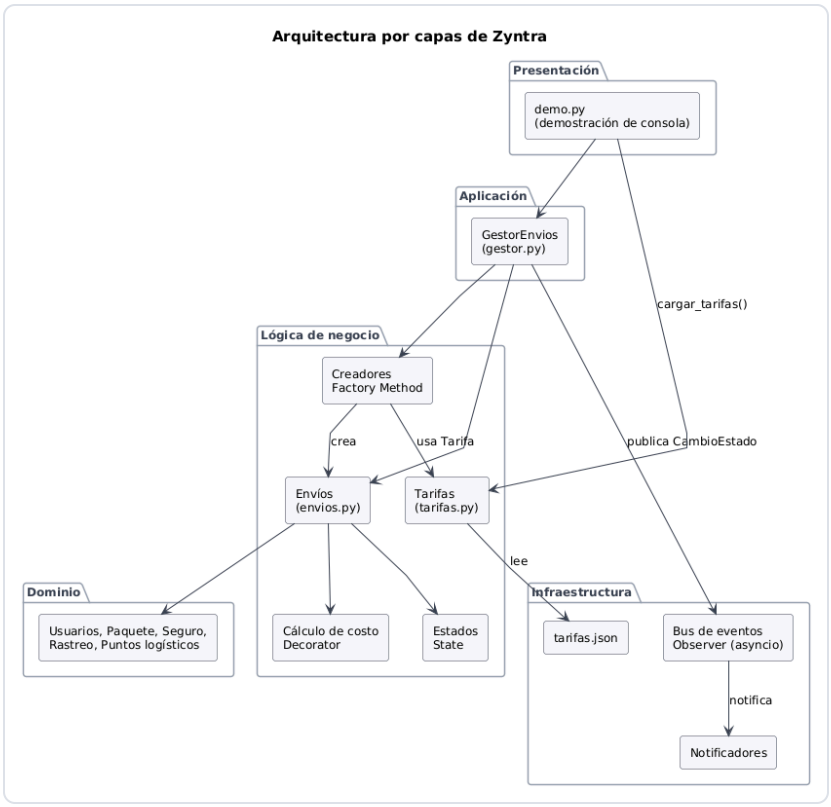


Figura 1. Arquitectura por capas.

Capa	Qué contiene	Responsabilidad
Presentación	`demo.py`	Demostración de consola; en una versión futura, la interfaz web.
Aplicación	`GestorEnvios`	Punto de entrada: registra envíos, cambia estados y publica eventos.
Lógica de negocio	Creadores, envíos, costos, estados, tarifas	Las reglas del negocio, organizadas con los patrones.
Dominio	Usuarios, paquetes, seguro, rastreo, puntos logísticos	Los datos del negocio y sus validaciones básicas.
Infraestructura	Bus de eventos, notificadores, `tarifas.json`	Comunicación asíncrona y configuración.

2.1 Modelo del dominio

Todo gira alrededor del envío: un cliente lo solicita en una sucursal, contiene uno o más paquetes, puede estar asegurado y va dejando registros de rastreo.

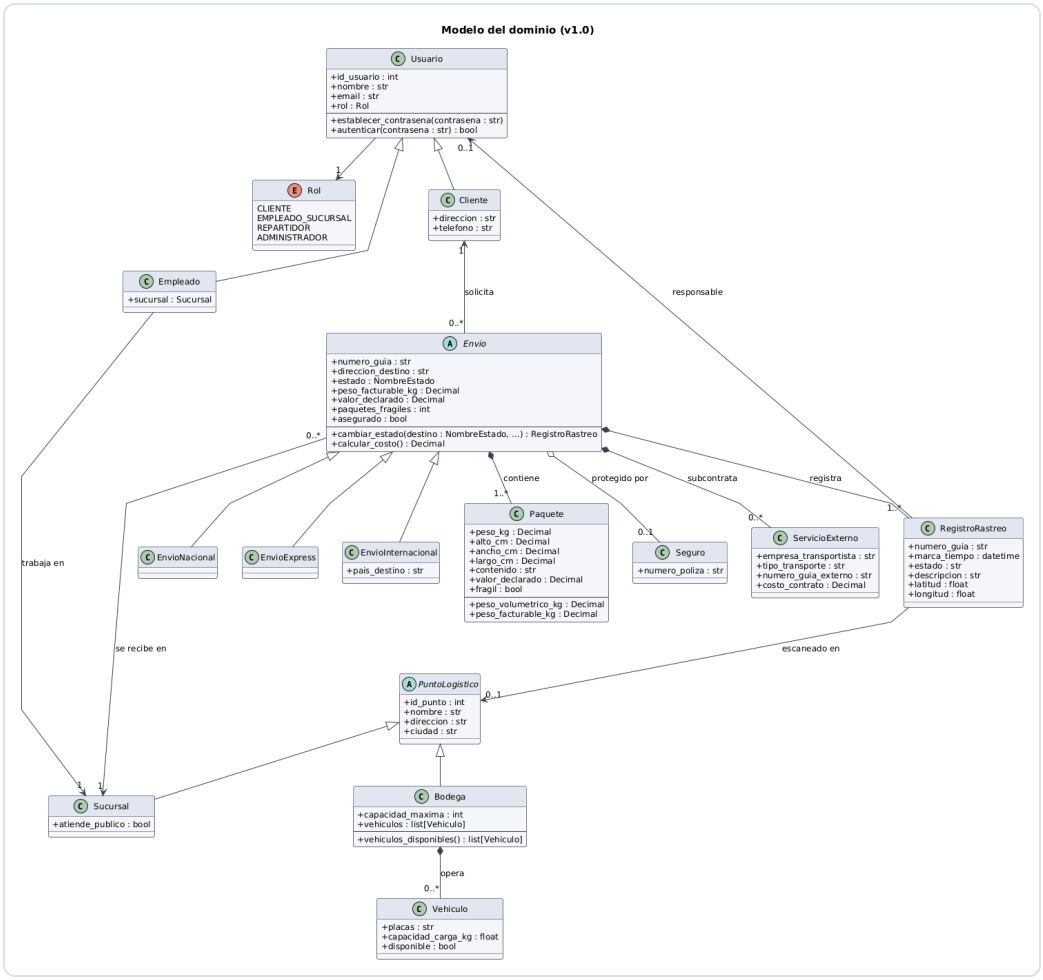


Figura 2. Modelo del dominio (v1.0).

- **Usuario, Cliente y Empleado.** Todo cliente y empleado es un usuario con correo y contraseña (guardada con hash PBKDF2). El empleado tiene un rol: empleado de sucursal, repartidor o administrador.
- **Envío.** Es abstracto y tiene tres tipos. Su peso y su valor declarado se calculan sumando sus paquetes; no se guardan dos veces.
- **Paquete.** Guarda peso, medidas, contenido, valor declarado y si es frágil. Se valida al crearse y calcula su peso facturable: el mayor entre el real y el volumétrico (largo × ancho × alto / 5000).
- **Seguro (0..1).** Es opcional y cubre el valor declarado del envío.
- **RegistroRastreo.** Cada movimiento con fecha, estado, descripción, responsable y, si aplica, el punto logístico o las coordenadas.
- **Puntos logísticos.** Sucursales (atienden al público) y bodegas (almacenan y operan vehículos) comparten nombre, dirección y ciudad.
- **Servicio externo.** Un tramo subcontratado a otra transportista, con su guía y el costo del contrato.

3 Punto de partida y deuda técnica

La v0.1 metía todo en una sola clase, `Envio`: guardaba los datos, calculaba el costo y controlaba los cambios de estado.

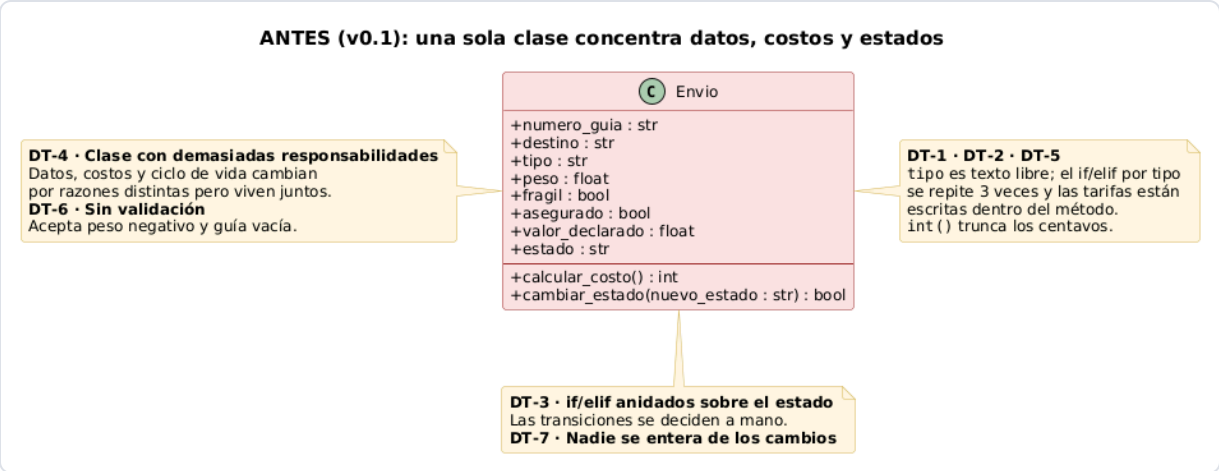


Figura 3. Diseño de partida (v0.1) con la deuda técnica señalada.

El código original se conserva en `legado/envio_legado.py` y en la etiqueta **v0.1**. Sus pruebas de caracterización fijan cómo se comporta, errores incluidos, para comprobar que la refactorización conserva lo que sí funcionaba.

```
def calcular_costo(self):
    if self.tipo == "nacional":
        costo = 80 + 15 * self.peso
    elif self.tipo == "internacional":
        costo = 950 + 280 * self.peso
    elif self.tipo == "express":
        costo = 150 + 25 * self.peso
    else:
        costo = 0

    if self.asegurado:
        if self.tipo == "nacional":
            costo += self.valor_declarado * 0.03
        elif self.tipo == "internacional":
            costo += self.valor_declarado * 0.05
        elif self.tipo == "express":
```

3.1 Problemas detectados

Clave	Problema	Consecuencia	Se resuelve con
DT-1	La decisión «nacional / internacional / express» se repite tres veces en `calcular_costo`.	Agregar un tipo obliga a editar cada `if/elif`.	Factory Method
DT-2	El tipo y el estado son textos sueltos.	`Envio(..., "Nacional", ...)` cuesta L 0 sin avisar (prueba `test_tipo_mal_escrito_no_falla`).	Factory Method, State (enums)
DT-3	Las transiciones se deciden con `if` anidados.	Cada estado nuevo vuelve más difícil entender las reglas.	State
DT-4	Una sola clase guarda datos, calcula costos y controla estados.	Un cambio en las tarifas puede romper el manejo de estados.	Decorator, State
DT-5	Las tarifas están escritas dentro del código.	Cambiar un precio implica buscarlo en medio de la lógica.	Tarifa + tarifas.json
DT-6	No se validan los datos de entrada.	Acepta peso −3 kg y guía vacía (prueba `test_acepta_peso_negativo`).	Factory Method, Paquete
DT-7	Nadie se entera cuando un envío cambia.	No se puede ofrecer rastreo en tiempo real.	Observer

Dos principios guían la solución: **una clase, una responsabilidad** (cada clase cambia por una sola razón) y **abierto a extensión, cerrado a modificación** (agregar algo nuevo no obliga a editar lo que ya existe).

4 Justificación de los patrones

Un patrón de diseño es una solución probada a un problema que se repite. Se eligió uno de cada familia del catálogo GoF, y cada uno responde a problemas concretos de la sección anterior.

Patrón	Familia	Resuelve	Archivo	Analogía
Factory Method	Creacional	DT-1, DT-2, DT-6	`creadores.py`	La empresa decide en el momento si usa camión o barco según el terreno.
Decorator	Estructural	DT-4	`costos.py`	Capas de ropa: un suéter sobre la camisa añade calor sin cambiar a la persona.
State	Comportamiento	DT-2, DT-3	`estados.py`	Semáforo: el objeto es el mismo, pero actúa distinto según su estado.
Observer	Comportamiento	DT-7	`eventos.py`	Suscripción a una revista: cuando sale la edición, llega sin pedirla.

DT-5 no necesita un patrón: se resuelve sacando las tarifas a `datos/tarifas.json`, que se carga en objetos `Tarifa`.

4.1 Factory Method

Qué hace. `EnvioCreator` define el paso de creación (`crear_envio`) y deja que cada subclase decida qué envío fabricar (`_fabricar`). Antes de crear, valida la guía (formato `ZYN-00000000`), el destino, que haya al menos un paquete y que un envío asegurado tenga valor declarado. Cada creador agrega sus reglas: el express admite hasta 30 kg y el internacional exige país de destino distinto de Honduras y valor declarado en cada paquete, porque lo pide la aduana.

Por qué aquí. El resto del sistema pide «un envío de tipo X» y no necesita saber qué clase se crea. El tipo deja de ser un texto: es la clase misma y el enum `TipoEnvio`. Para no mover el `if/elif` al momento de elegir el creador, se usa un registro: agregar un tipo es agregar un creador y una entrada.

```
CLASES_CREADOR: dict[TipoEnvio, type[EnvioCreator]] = {
    TipoEnvio.NACIONAL: EnvioNacionalCreator,
    TipoEnvio.INTERNACIONAL: EnvioInternacionalCreator,
    TipoEnvio.EXPRESS: EnvioExpressCreator,
}
```

Por qué no otras opciones. Una función con condiciones solo cambiaría el problema de lugar. Abstract Factory sirve para familias completas de objetos relacionados, y aquí hay una sola familia. Builder conviene para objetos con muchos pasos de armado; crear un envío es un solo paso con validación.

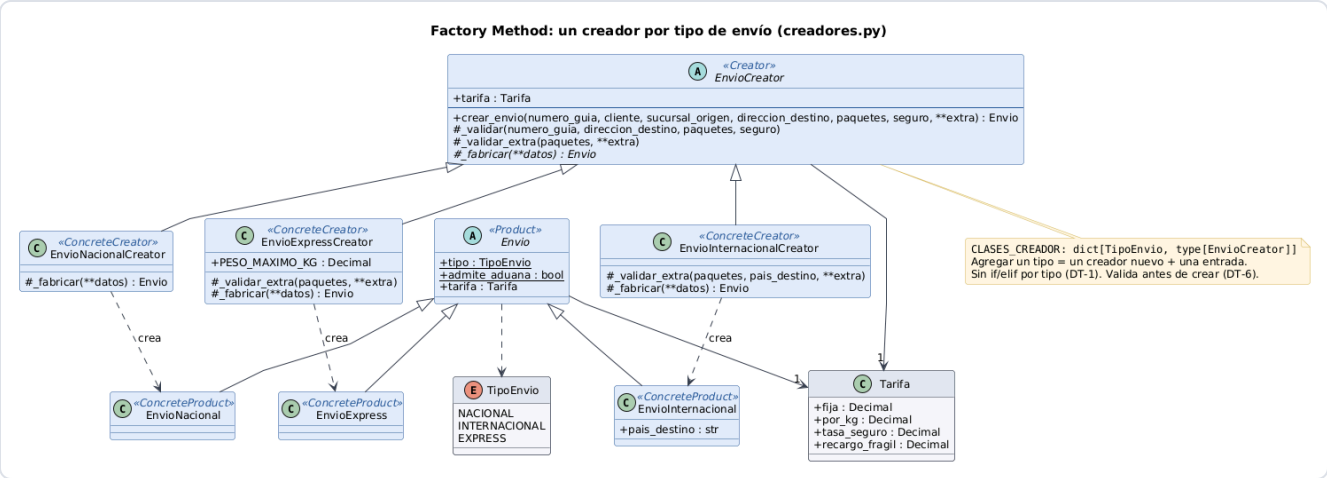


Figura 4. Factory Method aplicado a los envíos.

4.2 Decorator

Qué hace. `CostoBase` calcula la tarifa fija más el precio por kilo del peso facturable. `SeguroDecorator` y `ManejoFragilDecorator` envuelven esa calculadora y le suman su cargo solo cuando el envío lo necesita. Cada capa también aporta su línea al desglose que ve el cliente.

```
def armar_calculadora(envio: "Envio") → CalculadoraCosto:
    """Compone la cadena de decoradores según lo que necesita el envío."""
    tarifa = envio.tarifa
    calculadora: CalculadoraCosto = CostoBase(tarifa.fija, tarifa.por_kg,
                                              envio.peso_facturable_kg)
    if envio.asegurado:
        calculadora = SeguroDecorator(calculadora, tarifa.tasa_seguro, envio.valor_declarado)
    if envio.paquetes_fragiles:
        calculadora = ManejoFragilDecorator(calculadora, tarifa.recargo_fragil,
                                            envio.paquetes_fragiles)
    return calculadora
```

Por qué aquí. Cada envío se arma según lo que necesita: express asegurado, nacional con paquetes frágiles, internacional con ambos. Un extra nuevo, como «entrega en sábado», es una subclase nueva de `DecoradorCosto` que no toca el cálculo existente.

Por qué no herencia. Con 3 tipos y 2 extras habría que cubrir 3 × 4 = 12 combinaciones con subclases; con un tercer extra serían 3 × 8 = 24. Con Decorator cada extra es una sola clase.

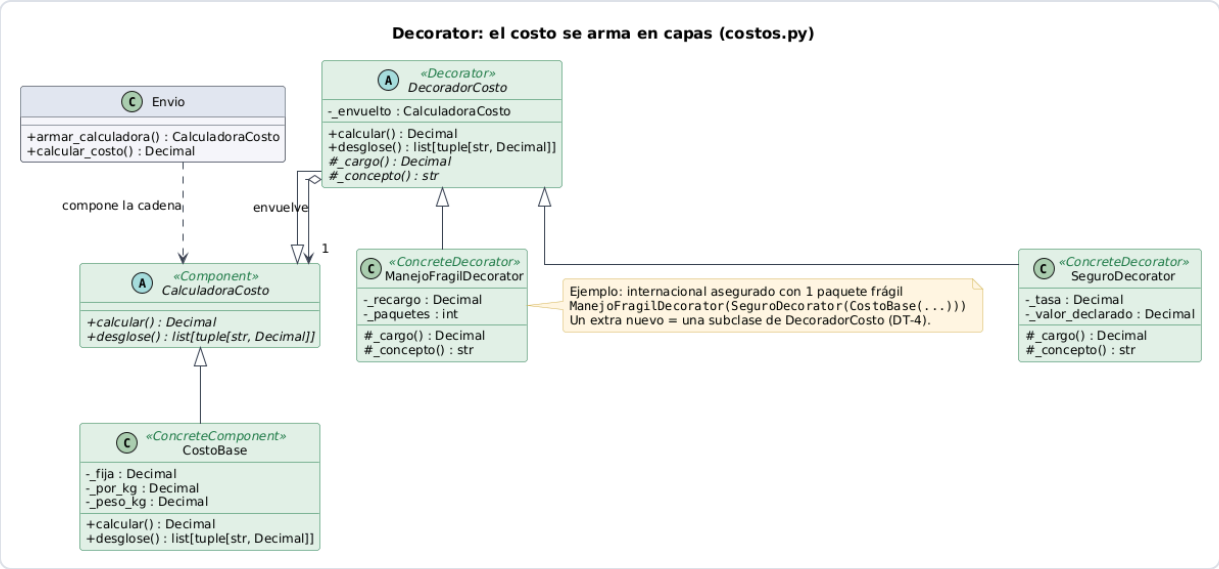


Figura 5. Decorator aplicado al cálculo del costo.

4.3 State

Qué hace. Cada estado es una clase que declara a qué estados puede pasar. El envío solo le pregunta a su estado actual si el cambio es válido; si no lo es, se lanza `TransicionInvalida` y nada cambia. Los nombres de estado son el enum `NombreEstado`, así que un error de escritura falla al instante.

```
class EnTransito(EstadoEnvio):
    nombre = NombreEstado.EN_TRANSITO
    destinos = frozenset({NombreEstado.EN_REPARTO, NombreEstado.RETENIDO_EN_ADUANA})

    def puede_ir_a(self, envio: "Envio", destino: NombreEstado) → bool:
        # Regla propia de este estado: solo los internacionales pasan por aduana.
        if destino is NombreEstado.RETENIDO_EN_ADUANA and not envio.admite_aduana:
            return False
        return super().puede_ir_a(envio, destino)
```

Por qué aquí. Como el semáforo, el envío es el mismo pero reacciona distinto según su estado. «Retenido en aduana» tiene una regla propia: solo aplica si el envío la admite, y solo `EnvioInternacional` declara `admite_aduana = True`. Esa regla vive en el estado al que pertenece.

Por qué no una tabla de transiciones. Bastaría si los estados solo tuvieran nombre, pero algunos tienen reglas propias (la de aduana). Con State cada regla queda en su clase y agregar un estado es una clase nueva más una transición.

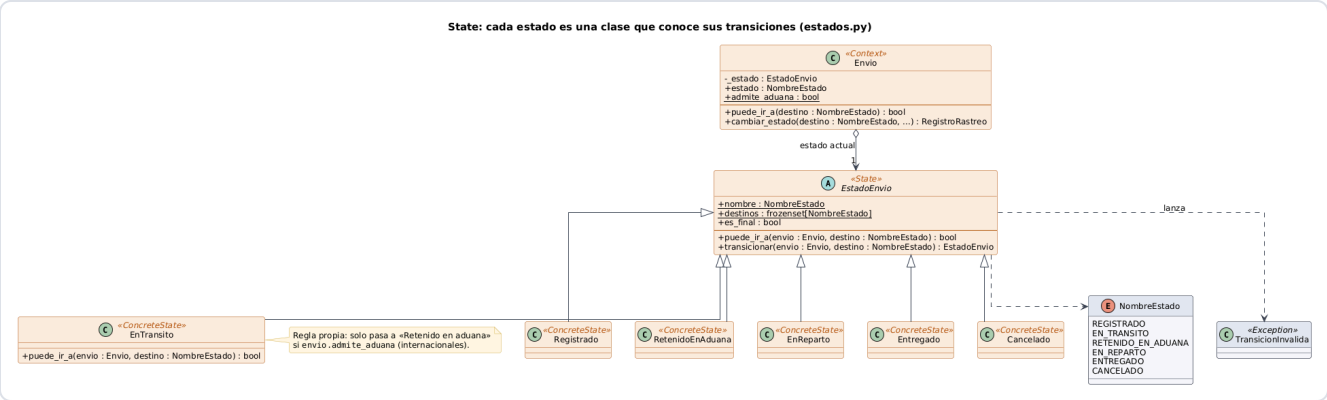


Figura 6. State aplicado al ciclo de vida del envío.

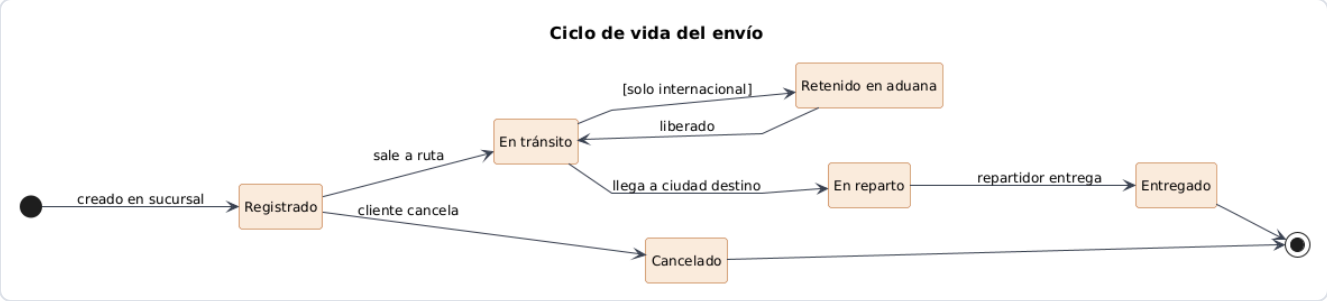


Figura 7. Estados posibles de un envío y sus transiciones.

4.4 Observer

Qué hace. `BusEventos` es el sujeto: recibe un evento `CambioEstado` por cada cambio y lo entrega a sus suscriptores, como el notificador de clientes y el panel de rastreo. El envío no conoce a quienes lo observan.

Por qué aquí. Resuelve DT-7: avisar sin acoplar. Agregar un canal nuevo (SMS, WhatsApp) es suscribir un observador nuevo, sin tocar envíos ni estados. Su diseño en tiempo real se explica en la sección 5.

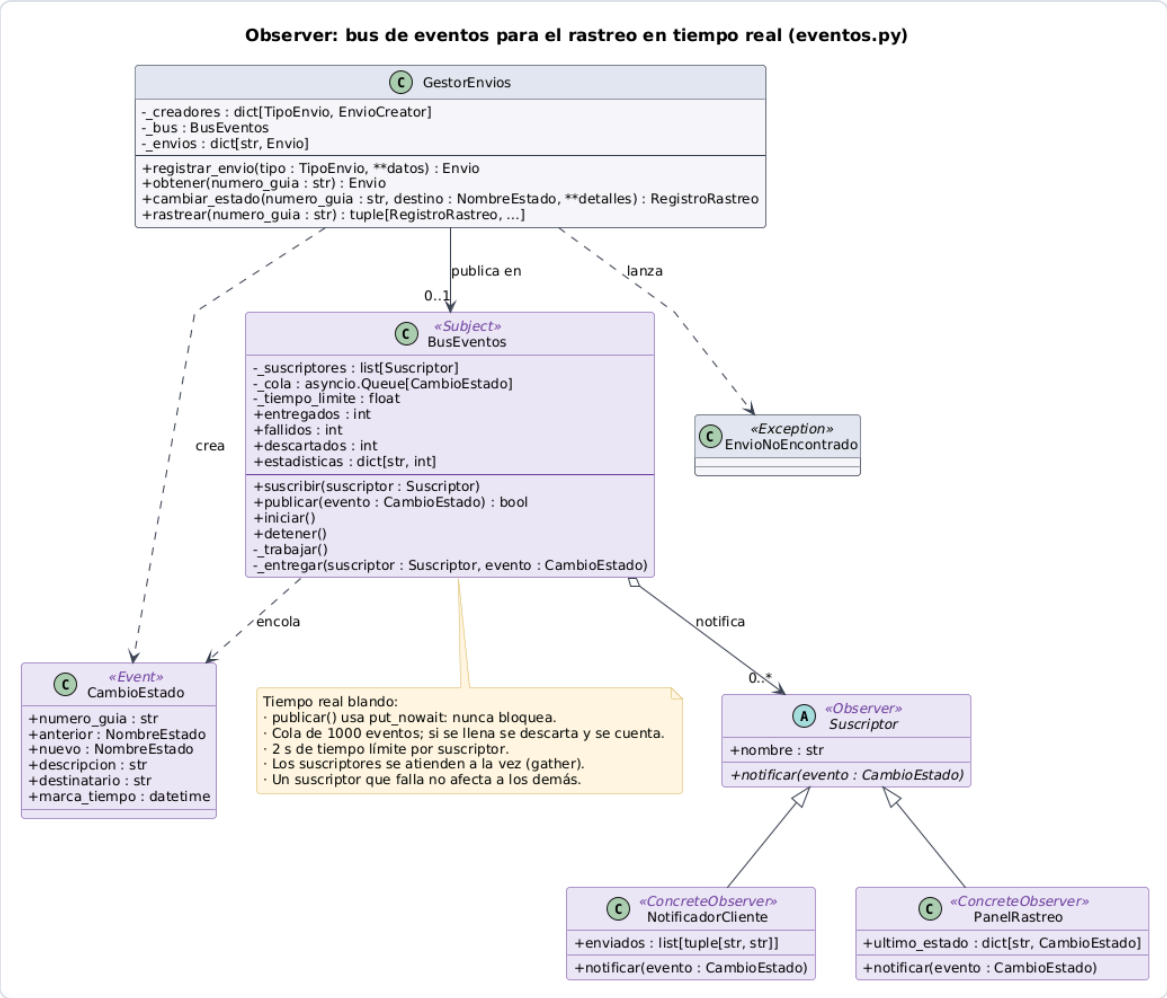


Figura 8. Observer: bus de eventos y sus suscriptores.

5 Diseño para el rastreo en tiempo real

Es tiempo real blando: si una notificación llega un par de segundos tarde no pasa nada grave, pero el sistema nunca debe quedarse detenido esperándola.

Regla	Cómo se cumple en el código
Publicar no bloquea	`publicar()` usa `put_nowait` y regresa de inmediato.
Tiempo límite por suscriptor	`wait_for` con 2 s; si se excede, se registra un aviso y se continúa.
Cola con límite	1000 eventos; si se llena, el evento se descarta y se cuenta en `descartados`.
Un fallo no afecta a los demás	Los suscriptores se atienden a la vez con `gather` y cada error se atrapa por separado.
El rastreo nunca se pierde	El registro se guarda de forma síncrona dentro de `cambiar_estado`, antes de publicar. Por el bus solo viajan los avisos.
Fácil de probar	El bus se crea fuera y se entrega a `GestorEnvios`; no es global.

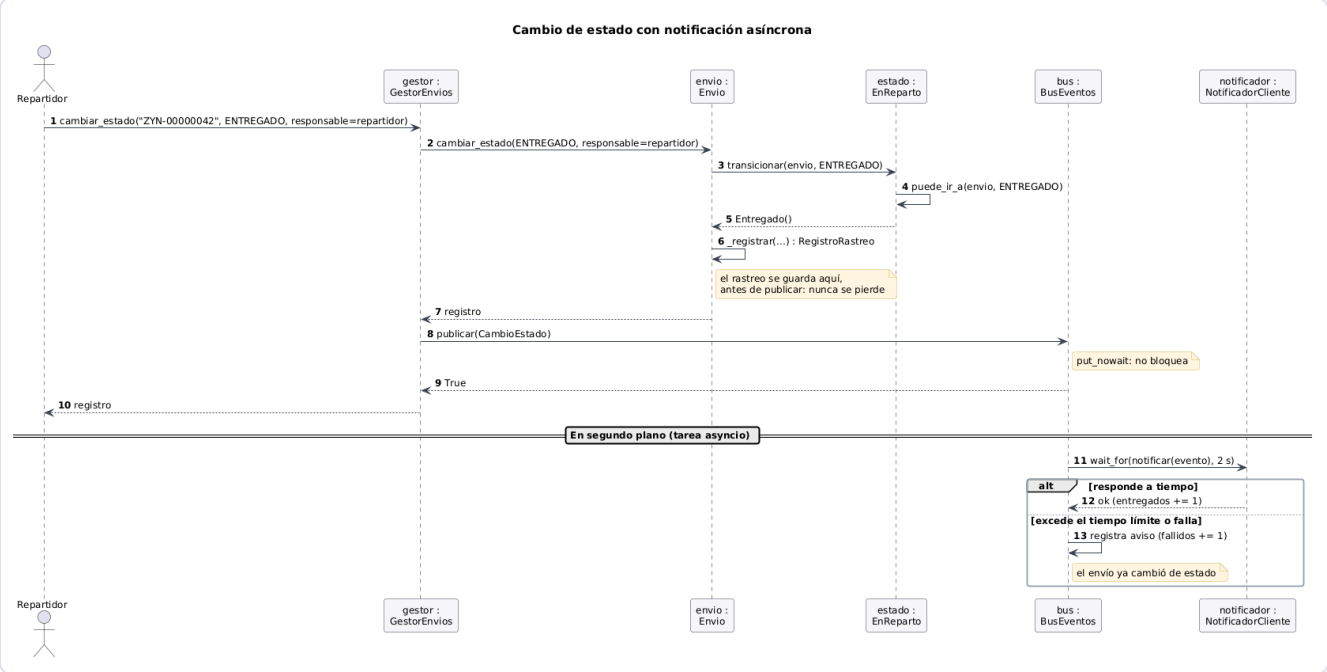


Figura 9. Cambio de estado con notificación en segundo plano.

> Este diseño es la base del «Módulo de respuesta» del Segundo Parcial (manejo de eventos concurrentes). Ahí se le agregarán reintentos y persistencia.

6 Comparativa UML: antes y después

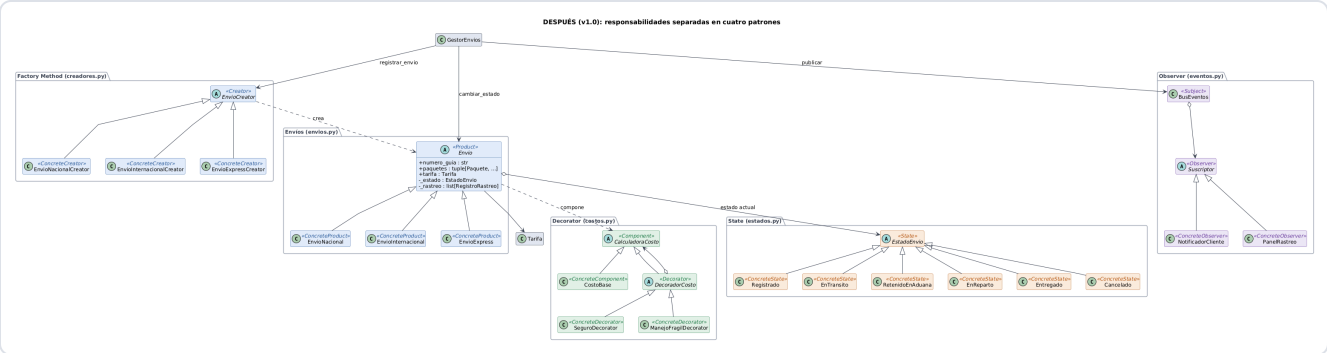


Figura 10. Diseño después de aplicar los patrones (comparar con la Figura 3).

Aspecto	Antes (v0.1)	Después (v1.0)
Clases	Una sola: `Envio`.	43 clases, cada una con una responsabilidad.
Tipo de envío	Texto libre; un error da costo 0.	Una clase por tipo y el enum `TipoEnvio`.
Creación	Sin validar: acepta peso negativo.	Un creador por tipo que valida antes de crear.
Costo	`if/elif` repetido; tarifas en el código; trunca centavos.	Costo base más capas; tarifas en JSON; `Decimal` con centavos.
Peso	Un solo número.	Suma de paquetes, con peso volumétrico.
Estados	`if` anidados sobre textos.	Una clase por estado; transición inválida lanza error.
Rastreo	No existía.	Registro por cada cambio y aviso asíncrono por el bus.
Agregar algo nuevo	Editar la clase central.	Agregar una clase nueva.
Pruebas	Ninguna.	48 pruebas automáticas.

Costo de la solución. Hay más clases y el diseño pide un poco más de lectura al inicio. A cambio, cada pieza es pequeña, se prueba por separado y crece sin romper lo existente.

7 Implementación y repositorio

El código está en Python 3.11+, sin librerías externas, en **github.com/TiarethE/zyntra**.

```
python -m zyntra.demo     # demostración de punta a punta
python -m unittest -v     # 48 pruebas automáticas
```

7.1 Tarifas

Tipo	Tarifa fija	Por kg	Seguro	Frágil (por paquete)
Nacional	L 80.00	L 15.00	3 %	L 50.00
Internacional	L 950.00	L 280.00	5 %	L 200.00
Express	L 150.00	L 25.00	4 %	L 100.00

Costo = tarifa fija + (por kg × peso facturable) + seguro (porcentaje del valor declarado, si está asegurado) + recargo por cada paquete frágil.

Ejemplo: envío internacional a Miami

Dos paquetes: café (3.5 kg, caja de 20 × 30 × 40 cm, valor L 1,500) y artesanía frágil (1 kg, valor L 800), asegurado. La caja de café pesa 4.8 kg volumétricos, así que se cobra ese peso. Este es el desglose que produce el código:

Concepto	Monto
Tarifa fija	L 950.00
Peso (5.8 kg)	L 1,624.00
Seguro (5 % de 2300)	L 115.00
Manejo frágil (1 paquete(s))	L 200.00
Total	**L 2,889.00**

7.2 Control de versiones

Cada patrón se desarrolló en su propia rama y se unió a `main` con sus pruebas pasando. La etiqueta **v0.1** marca el código de partida y **v1.0** la versión refactorizada. Los cambios se documentan en `CHANGELOG.md`.

```
*   6246a43 Merge docs/uml
|\
| * bb2dcaa Agrega diagramas UML generados a partir del código
|/
*   6ed82c4 Merge docs/readme-changelog
|\
| * df000a7 Documenta el proyecto en README y CHANGELOG
|/
*   2868a77 Merge feature/observer
|\
| * 110befb Implementa Observer: bus de eventos y notificaciones asíncronas
| * e2bc8ad Muestra el porcentaje del seguro sin ceros de más
|/
*   05a5b30 Merge feature/state
|\
| * f3093da Implementa State para el ciclo de vida del envío
|/
*   3c4d2ca Merge feature/decorator
|\
| * 48e3408 Implementa Decorator para el cálculo del costo
|/
*   195b7e1 Merge feature/factory-method
|\
| * d6bfb22 Implementa Factory Method para crear los tres tipos de envío
|/
*   41fdddb Merge feature/dominio-y-tarifas
|\
| * a621c2c Agrega clases del dominio, tarifas en JSON y pruebas de caracterización
|/
* 6eece08 Versión inicial: clase Envio con datos, costos y estados
```

7.3 Criterios de aceptación

Cada criterio tiene al menos una prueba automática que lo verifica.

Clave	Criterio	Prueba
CA-01	Los tres tipos se crean mediante su creador y rechazan datos inválidos.	`test_creadores.py · TestFactoryMethod, TestValidaciones`
CA-02	Los costos coinciden con la tabla para todas las combinaciones de tipo, seguro y fragilidad.	`test_costos.py · test_todas_las_combinaciones`
CA-03	La v1.0 da los mismos costos y transiciones que la v0.1.	`test_costos.py · TestMismoResultadoQueV01; test_estados.py · TestMismasTransicionesQueV01`
CA-04	Solo se permiten las transiciones del diagrama de estados.	`test_estados.py · test_solo_se_permiten_las_transiciones_del_diagrama`
CA-05	«Retenido en aduana» solo aplica a envíos internacionales.	`test_estados.py · test_aduana_solo_para_internacionales`
CA-06	Cada cambio de estado genera un registro de rastreo, aunque se descarten avisos.	`test_eventos.py · test_cola_llena_descarta_avisos_pero_no_rastreo`
CA-07	Si un notificador falla o tarda, el envío cambia de estado en menos de 10 ms.	`test_eventos.py · test_suscriptor_lento_se_corta_por_tiempo_limite, test_un_suscriptor_que_falla_no_afecta_a_los_demas`
CA-08	Los diagramas UML coinciden con el código.	`test_diagramas.py · TestDiagramasAlineados`

8 Conclusiones y limitaciones

Separar la creación, el costo, los estados y los avisos permitió que cada cambio futuro se resuelva agregando una clase y no editando una central. Los cuatro patrones responden a problemas reales del diseño de partida y se complementan: Factory Method entrega envíos válidos, Decorator calcula su costo, State controla su ciclo de vida y Observer avisa cada cambio. Las pruebas de caracterización demuestran que la v1.0 conserva los costos y transiciones de la v0.1, y corrige sus errores.

Limitaciones

- Los datos viven en memoria: no hay base de datos todavía.
- El bus de eventos no tiene reintentos ni persistencia; un aviso descartado no se recupera (el rastreo sí se conserva).
- Los notificadores simulan el envío de correos; no hay integración con un servicio real.
- El servicio express es solo nacional, y la asignación de vehículos a rutas queda fuera del alcance de esta etapa.
- No hay interfaz gráfica; la demostración es de consola.

A Referencias

- Fowler, M. (2018). *Refactoring: Improving the design of existing code* (2.ª ed.). Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Martin, R. C. (2003). *Agile software development: Principles, patterns, and practices*. Prentice Hall.
- Python Software Foundation. (2025). *asyncio: Asynchronous I/O*. https://docs.python.org/3/library/asyncio.html
- Universidad Anáhuac Mayab. (2026). *Tema 1.2: Catálogo de patrones de diseño (GoF)* [Material del curso]. Ingeniería de Software II.